PYTHON NOTES

Monika Barde ♡

1. INTRODUCTION

- Python is a high-level, interpreted, general-purpose programming language.
- Easy to learn and read.
- Supports multiple programming paradigms.
- Widely used in Web, Data Science, AI, Automation, etc.

2. VARIABLES & DATA TYPES

Variable: A container to store data values.
No need to declare type explicitly.

Types:

- int – Integer numbers
- float – Decimal numbers
- str – Text / String
- bool – True or False
- list – Ordered, mutable collection
- tuple – Ordered, immutable collection
- set – Unordered, unique items
- dict – Key-value pairs

Examples:

```
a = 10                  # int
b = 3.14                # float
c = "Hello"             # str
d = True                # bool
e = [1, 2, 3]           # list
f = (1, 2, 3)           # tuple
g = {1, 2, 3}           # set
h = {"name": "John", "age": 20}   # dict
```

3. OPERATORS

A. Arithmetic

```
+   -   *   /   //   %   **
```

B. Comparison

```
==   !=   >   <   >=   <=
```

C. Logical

```
and   or   not
```

D. Assignment

```
=   +=   -=   *=   /=   //=   %=   **=
```

Example:

```
a, b = 10, 3
a + b                   # 13
a // b                  # 3
a ** b                  # 1000
a > b                   # True
a == b                  # False
a > b and b < 5         # True
```

4. INPUT & OUTPUT

- input() – To take input from user (as string)
- print() – To display output

Example:

```
name = input("Enter your name: ")
print("Hello, " + name)
age = int(input("Enter age: "))
print("Age is:", age)
```

5. CONTROL FLOW

A. if-elif-else

```
x = 10
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

B. for loop

```
for i in range(5):
    print(i)        # 0 1 2 3 4
```

C. while loop

```
i = 0
while i < 5:
    print(i)
    i += 1
```

6. DATA STRUCTURES

A. List (mutable, ordered)

```
my_list = [1, 2, 3, "a"]
my_list.append(4)
my_list[0]          # 1
my_list[1:3]        # [2, 3]
```

B. Tuple (immutable, ordered)

```
my_tuple = (1, 2, 3)
my_tuple[0]         # 1
```

C. Set (unordered, unique)

```
my_set = {1, 2, 3, 2}
my_set.add(4)
my_set              # {1, 2, 3, 4}
```

D. Dictionary (key-value pairs)

```
person = {"name": "John", "age": 20}
person["name"]      # John
person["age"] = 21
```

7. FUNCTIONS

- Reusable block of code.
- Helps in modular programming.

Example:

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)       # 8
```

8. MODULES

- A file containing Python definitions and statements.
- Use 'import' to use modules.

Example:

```
import math
print(math.sqrt(16))    # 4.0
print(math.pi)          # 3.14159
```

9. STRING (str)

- Sequence of characters.
- Immutable.

Examples:

```
s = "Hello, Python"
s[0]                # 'H'
s[7:]               # 'Python'
s.lower()           # 'hello, python'
s.replace("Python", "World")
s.split(',')        # ['Hello', 'Python']
```

10. FILE HANDLING

- Used to read/write files.
- Modes: 'r' (read), 'w' (write), 'a' (append)

Example (Write):

```
f = open("file.txt", "w")
f.write("Hello\n")
f.write("Python is fun!")
f.close()
```

Example (Read):

```
f = open("file.txt", "r")
data = f.read()
print(data)
f.close()
```

11. EXCEPTIONS (ERROR HANDLING)

- Handle runtime errors using try-except.

Example:

```
try:
    x = int(input("Enter a number: "))
    print(10 / x)
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid input!")
finally:
    print("Execution completed.")
```

12. BUILT-IN FUNCTIONS (Common)

Function	Description
print()	Display output
input()	Take input
len()	Length of object
type()	Type of object
int(), float()	Convert to number
str()	Convert to string
list(), tuple()	Convert to list/tuple
max(), min()	Maximum / Minimum
sum()	Sum of items

13. COMMENTS

- Single line : `# This is a comment`
- Multi-line :

```
"""                         # Good code
This is a                   # is well
multi-line comment          # commented
"""
```

★ TIPS: Practice regularly, build small projects, and refer official Python documentation. Monika Barde ♡

PYTHON NOTES

Monika Barde

14. LOOPS

A. for loop

- Used to iterate over a sequence (list, tuple, string, range).

Syntax:

```
for variable in sequence:
    # code to execute
```

Example:

```
for i in range(1, 6):
    print(i)          # 1 2 3 4 5
```

B. while loop

- Repeats a block of code while condition is True.

Syntax:

```
while condition:
    # code to execute
```

Example:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

C. loop control statements

- `break` : Exit the loop
- `continue` : Skip current iteration
- `pass` : Do nothing (placeholder)

Example:

```
for i in range(1, 6):
    if i == 3:
        continue
    if i == 5:
        break
    print(i)          # 1 2 4
```

15. FUNCTIONS (Advanced)

A. Types of Functions

- Built-in functions
- User-defined functions
- Lambda (anonymous) functions

B. Default Argument

```
def greet(name, msg="Hello"):
    print(msg, name)
greet("Monika")      # Hello Monika
```

C. Keyword Argument

```
def add(a, b):
    return a + b
print(add(b=3, a=5))     # 8
```

D. Arbitrary Arguments

*args (non-keyword)

```
def sum_all(*args):
    return sum(args)
print(sum_all(1, 2, 3, 4))  # 10
```

**kwargs (keyword)

```
def info(**kwargs):
    for k, v in kwargs.items():
        print(k, ":", v)
info(name="Monika", age=20)
```

16. RECURSION

- A function that calls itself.
- Used for problems like factorial, Fibonacci, etc.

Example: Factorial

```
def fact(n):
    if n == 0 or n == 1:
        return 1
    return n * fact(n - 1)
print(fact(5))          # 120
```

17. LIST COMPREHENSION

- Concise way to create lists.

Syntax:

```
[expression for item in iterable
  if condition]
```

Example:

```
squares = [x**2 for x in range(1, 6)]
print(squares)    # [1, 4, 9, 16, 25]
```

18. LAMBDA FUNCTION

- Small anonymous function.
- Used with filter(), map(), reduce().

Syntax:

```
lambda arguments: expression
```

Example:

```
add = lambda a, b: a + b
print(add(3, 4))       # 7
```

19. MAP, FILTER, REDUCE

- `map()` : Applies function to all items.
- `filter()` : Filters items based on condition.
- `reduce()` : Applies rolling computation.

Example:

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x**2, numbers))
evens = list(filter(lambda x: x % 2 == 0, numbers))
sum_all = reduce(lambda x, y: x + y, numbers)

print(squares)    # [1, 4, 9, 16, 25]
print(evens)      # [2, 4]
print(sum_all)    # 15
```

20. OOPs CONCEPTS

A. Class and Object

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print(self.name, self.age)

s = Student("Monika", 20)
s.display()              # Monika 20
```

B. Inheritance C. Polymorphism

D. Encapsulation E. Abstraction

(Detailed in OOPs Notes)

21. FILE HANDLING (Advanced)

Modes:

- `'r'` : read
- `'w'` : write (overwrites)
- `'a'` : append
- `'r+'` : read and write
- `'w+'` : write and read

Example:

```
with open("data.txt", "w") as f:
    f.write("Hello Python\n")
    f.write("Keep Learning!\n")

with open("data.txt", "r") as f:
    print(f.read())
```

22. EXCEPTION HANDLING (Advanced)

- Handle specific exceptions.

Example:

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ValueError:
    print("Please enter a valid number.")
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Result:", result)
finally:
    print("Execution finished.")
```

23. MODULES & PACKAGES

- Module: A file containing Python code.
- Package: Collection of modules.

Example:

```
import math
print(math.sqrt(16))     # 4.0
print(math.pi)           # 3.14159
```

24. ITERATORS & GENERATORS

A. Iterator

- Object with `__iter__()` and `__next__()`.

B. Generator

- Uses `'yield'` to produce values.

Example:

```
def count(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for x in count(3):
    print(x)          # 1 2 3
```

25. USEFUL TIPS

- Use meaningful variable names.
- Follow PEP 8 style guide.
- Use functions to avoid code repetition.
- Comment your code.
- Practice regularly!

⭐ **TIPS:** Practice regularly, build small projects, and refer official Python documentation.

Monika Barde ♡

PYTHON NOTES

Monika Barde

26. REGULAR EXPRESSIONS

- Used for pattern matching in strings.
- Module: re

Example:

```
import re
pattern = r"\d+"        # digits
text = "I have 123 apples"
match = re.search(pattern, text)
print(match.group())    # 123
```

27. DATE & TIME

- Module: datetime
- Work with dates and times.

Example:

```
from datetime import datetime
now = datetime.now()
print(now)          # current date & time
print(now.strftime("%d-%m-%Y"))
                    # formatted date
```

28. JSON (JavaScript Object Notation)

- Lightweight data interchange format.
- Module: json

Example:

```
import json
data = {"name": "Monika", "age": 20}
js = json.dumps(data)
print(js)   # {"name": "Monika", "age": 20}
py = json.loads(js)
print(py["name"])    # Monika
```

29. UNIT TESTING

- Module: unittest
- Used for testing code.

Example:

```
import unittest
def add(a, b):
    return a + b

class TestAdd(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)

if __name__ == "__main__":
    unittest.main()
```

30. VIRTUAL ENVIRONMENT

- Isolated environment for Python projects.
- Commands:

```
python -m venv myenv          # create env
myenv\Scripts\activate        # activate (Win)
source myenv/bin/activate     # activate (Mac/Linux)
deactivate                    # deactivate
```

31. PIP & PACKAGES

- pip is the package installer for Python.
- Commands:

```
pip install package           # install
pip uninstall package         # uninstall
pip list                      # list installed
pip freeze > requirements.txt # save
pip install -r requirements.txt  # install from file
```

32. DECORATORS

- Modify behavior of functions.
- Use @decorator_name before function.

Example:

```
def my_decorator(func):
    def wrapper():
        print("Before function")
        func()
        print("After function")
    return wrapper

@my_decorator
def say_hello():
    print("Hello Monika!")

say_hello()
```

33. GENERATOR FUNCTIONS

- Use yield instead of return.
- Memory efficient.

Example:

```
def count(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for x in count(5):
    print(x)     # 1 2 3 4 5
```

34. CONTEXT MANAGER

- Used with 'with' statement.
- Ensures proper resource management.

Example:

```
with open("file.txt", "r") as f:
    data = f.read()
    print(data)
# file is automatically closed
```

35. TYPE HINTING

- Specify expected types of variables, function arguments and return values.

Example:

```
from typing import List

def add(a: int, b: int) -> int:
    return a + b

def total(nums: List[int]) -> int:
    return sum(nums)
```

36. NAMED TUPLE

- Tuple with named fields.

Example:

```
from collections import namedtuple

Person = namedtuple("Person", ["name", "age"])

p = Person("Monika", 20)
print(p.name)    # Monika
print(p.age)     # 20
```

37. ENUMERATION

- Add names to integer constants.
- Module: enum

Example:

```
from enum import Enum

class Color(Enum):
    RED = 1
    GREEN = 2
    BLUE = 3

print(Color.RED)    # Color.RED
print(Color(2))     # Color.GREEN
```

38. ZIP, ENUMERATE, ANY, ALL

A. zip() - Combines iterables.

Example:

```
names = ["A", "B", "C"]
ages = [20, 21, 22]
print(list(zip(names, ages)))
# [ 'A', 20), ('B', 21), ('C', 22)]
```

B. enumerate() - Adds counter to iterable.

Example:

```
for i, name in enumerate(names, start = 1):
    print(i, name)
# 1 A\n # 2 B\n # 3 C
```

C. any() - True if any element is True.

Example:

```
print(any([False, False, True]))    # True
```

D. all() - True if all elements are True.

Example:

```
print(all([True, True, True]))      # True
```

39. COMMON STRING METHODS

Example:

```
s = "  Hello Python  "
s.upper()              # HELLO PYTHON
s.lower()              # hello python
s.strip()              # Hello Python
s.lstrip()             # Hello Python
s.rstrip()             #   Hello Python
s.replace("Python", "World")  # Hello World
s.split()              # ['Hello', 'Python']
s.startswith("He")     # True
s.endswith("on")       # True
s.find("lo")           # 3
s.count("l")           # 3
```

BONUS - MAGIC METHODS (DUNDER)

Example of __str__ and __len__

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f"Book: {self.title}"

    def __len__(self):
        return self.pages

b = Book("Python Basics", 150)
print(b)        # Book: Python Basics
print(len(b))   # 150
```

⭐ TIPS: Practice regularly, build small projects, and refer official Python documentation.

Monika Barde ♡

PYTHON NOTES

Monika Barde

40. LIST METHODS

Example:

```
lst = [1, 2, 3]
lst.append(4)          # [1, 2, 3, 4]
lst.extend([5, 6])     # [1, 2, 3, 4, 5, 6]
lst.insert(1, 10)      # [1, 10, 2, 3, 4, 5, 6]
lst.remove(2)          # [1, 10, 3, 4, 5, 6]
lst.pop()              # 6 -> [1, 10, 3, 4, 5]
lst.clear()            # []
lst.index(10)          # 1
lst.count(3)           # 1
lst.sort()             # [1, 3, 4, 5, 10]
lst.reverse()          # [10, 5, 4, 3, 1]
lst.copy()             # shallow copy
```

41. TUPLE METHODS

Example:

```
t = (1, 2, 3, 2)
t.count(2)        # 2
t.index(2)        # 1
# Note: Tuples are immutable (no add/remove)
```

42. DICTIONARY METHODS

Example:

```
d = {'a': 1, 'b': 2}
d['c'] = 3              # {'a': 1, 'b': 2, 'c': 3}
d.get('a')              # 1
d.get('x', 'Not Found') # Not Found
d.keys()                # dict_keys(['a', 'b', 'c'])
d.values()              # dict_values([1, 2, 3])
d.items()               # dict_items([('a', 1),...])
d.update({'b': 20})     # {'a': 1, 'b': 20, 'c': 3}
d.pop('a')              # 1 -> {'b': 20,'c':3}
d.popitem()             # ('c', 3)
d.clear()               # {}
```

43. SET METHODS

Example:

```
s = {1, 2, 3}
s.add(4)                    # {1,2,3,4}
s.update({4, 5, 6})         # {1,2,3,4,5,6}
s.remove(2)                 # {1,3,4,5,6}
s.discard(10)               # no error if not present
s.pop()                     # removes random item
s.clear()                   # set()
s.union({6, 7})             # {1,2,3,4,5,6,7}
s.intersection({3,4,8})     # {3,4}
s.difference({3,4})         # {1,2,5,6}
s.symmetric_difference({1, 2, 10}) # {3,4,5,6,10}
```

44. COMPREHENSIONS

Example:

```
List Comprehension
squares = [x*x for x in range(1, 6)]
                        # [1, 4, 9, 16, 25]
Dictionary Comprehension
d = {x: x*x for x in range(1, 4)}
                    # {1:1, 2:4, 3:9}
Set Comprehension
s = {x for x in range(1, 6) if x % 2 == 0}
                    # {2, 4}
```

45. ENUMERATE, ZIP, REVERSED, SORTED

Example:

```
nums = ['a', 'b', 'c']
for i, val in enumerate(nums):
    print(i, val)       # 0 a / 1 b / 2 c

names = ['A', 'B', 'C']
ages  = [20, 21, 22]
z = list(zip(names, ages))
      # [('A', 20),('B', 21),('C', 22)]

rev = list(reversed(nums))    # ['c', 'b', 'a']

sorted_nums = sorted([3, 1, 2])  # [1, 2, 3]
sorted_desc = sorted([3, 1, 2], reverse=True)
                                 # [3, 2, 1]
```

46. LAMBDA & HIGHER ORDER FUNCTIONS

- Lambda: small anonymous function.
- Functions that take other functions or return them.

Example:

```
add = lambda a, b: a + b
print(add(2, 3))      # 5

nums = [1, 2, 3, 4]
squares = list(map(lambda x: x*x, nums))
                                # [1,4,9, 16]
evens = list(filter(lambda x: x % 2 == 0, nums))
                                # [2, 4]

from functools import reduce
product = reduce(lambda x, y: x*y, nums)  # 24
```

47. PASS, DEL & GLOBAL KEYWORD

Example:

```
def func():
    pass  # placeholder

x = 10
def modify():
    global x
    x = 20

modify()
print(x)     # 20

lst = [1, 2, 3]
del lst[1]        # [1, 3]
del lst           # lst is deleted
```

48. ASSERT STATEMENT

- Used for debugging.
- Raises AssertionError if condition is False.

Example:

```
x = 10
assert x > 0, "x must be positive"
print("OK")
```

49. DOCSTRINGS

- Used to document modules, classes, functions.

Example:

```
def cube(n):
    """Return cube of a number."""
    return n ** 3

print(cube.__doc__)   # Return cube of a number.
help(cube)
```

50. REGULAR EXPRESSIONS (Advanced)

- Powerful tool for pattern matching.
- Module: re

Example:

```
import re
text = "Contact: 987-654-3210"
pattern = r"\d{3}-\d{3}-\d{4}"
match = re.search(pattern, text)
print(match.group())   # 987-654-3210
```

51. LOGGING

- Used to track events in a program.
- Levels: DEBUG, INFO, WARNING, ERROR, CRITICAL

Example:

```
import logging
logging.basicConfig(level=logging.INFO,
                    format='%(levelname)s: %(message)s')
logging.debug('This is debug')
logging.info('This is info')
logging.warning('This is warning')
logging.error('This is error')
logging.critical('This is critical')
```

52. WORKING WITH CSV FILES

Example:

```
import csv

with open('data.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Name', 'Age'])
    writer.writerow(['Monika', '20'])

with open('data.csv', 'r') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

53. WORKING WITH JSON FILES

Example:

```
import json
data = {'name': 'Monika', 'age': 20}
# Write JSON
with open('data.json', 'w') as f:
    json.dump(data, f, indent=4)
# Read JSON
with open('data.json', 'r') as f:
    d = json.load(f)
    print(d)
```

54. ARGPARSE (COMMAND LINE ARGUMENTS)

Example:

```
import argparse
parser = argparse.ArgumentParser(description='Demo')
parser.add_argument('--name', type=str, help='Your name')
parser.add_argument('--age', type=int, help='Your age')
args = parser.parse_args()
print(args.name, args.age)
```

55. TIME COMPLEXITY GUIDE (Basic)

Example:

• Constant Time	O(1)
• Logarithmic Time	O(log n)
• Linear Time	O(n)
• Linearithmic Time	O(n log n)
• Quadratic Time	O(n^2)
• Cubic Time	O(n^3)
• Exponential Time	O(2^n)

★ TIPS: Practice regularly, build small projects, and refer official Python documentation.

Monika Barde

PYTHON NOTES

Monika Barde

60. DATA CLASSES (dataclasses)

- Simplify class creation for storing data.
- Module: dataclasses

Example:

```
from dataclasses import dataclass
@dataclass
class Student:
    name: str
    age: int
s = Student("Monika", 20)
print(s)
# Student(name='Monika', age=20)
```

61. PATHLIB MODULE

- Object-oriented way to handle file paths.
- Module: pathlib

Example:

```
from pathlib import Path
p = Path("/home/user/data/file.txt")
print(p.name)          # file.txt
print(p.parent)        # /home/user/data
print(p.suffix)        # .txt
print(p.exists())      # True / False
```

62. SUBPROCESS MODULE

- Run new programs and manage processes.
- Module: subprocess

Example:

```
import subprocess
result = subprocess.run(["ls", "-l"],
          capture_output=True, text=True)
print(result.stdout)
```

63. ARGPARSE MODULE

- Parse command line arguments.
- Module: argparse

Example:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--name", type=str)
parser.add_argument("--age", type=int)
args = parser.parse_args()
print(args.name, args.age)
```

64. CONFIGPARSER MODULE

- Read and write configuration files.
- Module: configparser

Example:

```
import configparser
config = configparser.ConfigParser()
config['DEFAULT'] = {'server': 'local'}
config['database'] = {'user': 'admin'}
with open('config.ini', 'w') as f:
    config.write(f)
```

65. SHELL OPERATIONS (os module)

- Common OS operations.

Example:

```
import os
os.system('echo Hello')      # run shell cmd
os.getenv('PATH')            # env variable
os.chdir('/home/user')       # change dir
os.getcwd()                  # current dir
```

66. STATISTICS MODULE

- Mathematical statistics functions.
- Module: statistics

Example:

```
import statistics
data = [2, 4, 4, 4, 5, 5, 7, 9]
print(statistics.mean(data))      # 5.0
print(statistics.median(data))    # 4.5
print(statistics.mode(data))      # 4
```

67. HEAPQ MODULE

- Implements heap queue algorithm.
- Module: heapq

Example:

```
import heapq
nums = [4, 1, 7, 3, 8, 5]
heapq.heapify(nums)
heapq.heappush(nums, 2)
print(heapq.heappop(nums))    # 1
print(nums)          # [2, 3, 7, 4, 8, 5]
```

68. BISECT MODULE

- Binary search functions.
- Module: bisect

Example:

```
import bisect
a = [1, 3, 5, 7, 9]
print(bisect.bisect_left(a, 4))    # 2
print(bisect.insort(a, 4))
print(a)             # [1, 3, 4, 5, 7, 9]
```

69. SCHEDULING TASKS (schedule)

- Simple job scheduling for Python.
- Module: schedule

Example:

```
import schedule, time
def job():
    print("Task is running...")
schedule.every(10).seconds.do(job)
while True:
    schedule.run_pending()
    time.sleep(1)
```

70. THREADING MODULE

- Run tasks concurrently using threads.

Example:

```
import threading, time
def task():
    for i in range(3):
        print(f"Thread: {i}")
        time.sleep(1)
t = threading.Thread(target=task)
t.start()
t.join()
```

71. MULTIPROCESSING MODULE

- Run tasks in parallel using processes.

Example:

```
import multiprocessing as mp
def square(x):
    return x * x
with mp.Pool(4) as pool:
    result = pool.map(square, [1, 2, 3, 4])
print(result)    # [1, 4, 9, 16]
```

72. QUEUE MODULE

- Thread-safe queue implementation.
- Module: queue

Example:

```
from queue import Queue
q = Queue()
q.put(10)
q.put(20)
print(q.get())    # 10
print(q.get())    # 20
```

73. PRIORITY QUEUE

- Implemented using heapq.

Example:

```
import heapq
pq = []
heapq.heappush(pq, (2, 'low'))
heapq.heappush(pq, (1, 'high'))
print(heapq.heappop(pq))    # (1, 'high')
```

74. CACHING (functools.lru_cache)

- Cache results of expensive function calls.

Example:

```
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)
print(fib(30))    # fast with caching
```

75. DEBUGGING (pdb)

- Python debugger.

Example:

```
import pdb
x = 10
y = 0
pdb.set_trace()    # set breakpoint
z = x / y
```

76. PROFILE CODE (timeit)

- Measure small code snippets.

Example:

```
import timeit
code = "sum([i*i for i in range(100)])"
t = timeit.timeit(code, number=1000)
print(t)
```

77. DOCSTRINGS & DOCUMENTATION

- Write docstrings using triple quotes.

Example:

```
def add(a, b):
    """Return sum of two numbers."""
    return a + b
help(add)      # show documentation
```

78. TYPE CHECKING (mypy)

- Static type checker.

Example:

```
# Install:  pip install mypy
# Run:  mypy filename.py
```

TIPS: Practice regularly, build small projects, and refer official Python documentation.

Monika Barde

PYTHON NOTES

Monika Barde

60. PATHLIB MODULE

- Object-oriented approach to file system paths.
- Module: pathlib

Example:

```
from pathlib import Path
p = Path("example.txt")
print(p.exists())       # check if exists
print(p.read_text())    # read file
```

61. SUBPROCESS MODULE

- Run external commands.
- Module: subprocess

Example:

```
import subprocess
result = subprocess.run(["dir"],
                        capture_output=True,
                        text=True)
print(result.stdout)
```

62. STATISTICS MODULE

- Mathematical statistics functions.
- Module: statistics

Example:

```
import statistics
data = [10, 20, 30, 40, 50]
print(statistics.mean(data))      # 30
print(statistics.median(data))    # 30
print(statistics.mode(data))      # 10
```

63. HEAPQ MODULE

- Implements heap queue algorithm.
- Module: heapq

Example:

```
import heapq
nums = [5, 1, 8, 3]
heapq.heapify(nums)
print(heapq.heappop(nums))    # 1
```

64. COUNTER (collections)

- Count hashable objects.

Example:

```
from collections import Counter
words = ['a', 'b', 'a', 'c', 'b', 'a']
count = Counter(words)
print(count)    # Counter({'a': 3, 'b': 2, 'c': 1})
```

65. DEQUE (collections)

- Double ended queue.

Example:

```
from collections import deque
dq = deque([1, 2, 3])
dq.append(4)       # add right -> [1,2,3,4]
dq.appendleft(0)   # add left  -> [0,1,2,3,4]
dq.pop()           # remove right -> 4
dq.popleft()       # remove left  -> 0
```

66. ITETOOLS MODULE

- Functions creating iterators for efficient loops.

Example:

```
import itertools
nums = [1, 2, 3]
print(list(itertools.permutations(nums, 2)))
# [(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
```

67. F-STRINGS (Formatted Strings)

- Easy way to format strings.

Example:

```
name = "Monika"
age = 20
msg = f"Hello {name}, you are {age} years old."
print(msg)  # Hello Monika, you are 20 years old.
```

68. STRING FORMATTING (format())

Example:

```
name = "Monika"
age = 20
msg = "My name is {} and I am {} years old."
print(msg.format(name, age))
```

69. LIST COMPREHENSION (Advanced)

- Create lists in a single line.

Example:

```
squares = [x*x for x in range(1, 6) if x % 2 == 0]
print(squares)   # [4, 16]
```

70. DICTIONARY COMPREHENSION

Example:

```
nums = [1, 2, 3, 4]
square_dict = {x: x*x for x in nums}
print(square_dict)   # {1:1, 2:4, 3:9, 4:16}
```

71. SET COMPREHENSION

Example:

```
nums = [1, 2, 2, 3, 4, 4, 5]
unique = {x for x in nums}
print(unique)      # {1, 2, 3, 4, 5}
```

72. SLICING (Advanced)

Example:

```
s = "Python Programming"
print(s[0:6])      # Python
print(s[::-1])     # gnimmargorP nohtyP
print(s[7:])       # Programming
```

73. SORTING TECHNIQUES

Example:

```
nums = [5, 2, 9, 1]
print(sorted(nums))                  # [1, 2, 5, 9]
print(sorted(nums, reverse=True))    # [9, 5, 2, 1]
nums.sort()
print(nums)                          # [1, 2, 5, 9]
```

74. LARGEST & SMALLEST

Example:

```
nums = [10, 5, 20, 3, 15]
print(max(nums))    # 20
print(min(nums))    # 3
print(sum(nums))    # 53
```

75. ANY() & ALL()

Example:

```
nums = [2, 4, 6, 8]
print(all(x % 2 == 0 for x in nums))   # True
print(any(x > 5 for x in nums))        # True
```

76. ASSERTION

- Check if a condition is True, else raise error.

Example:

```
x = -5
assert x >= 0, "x must be non-negative"
# AssertionError: x must be non-negative
```

77. DOCSTRINGS

- Documentation string for functions, classes, modules.

Example:

```
def greet(name):
    """This function greets a person."""
    return f"Hello, {name}!"

help(greet)   # shows documentation
```

78. LRU CACHE (functools)

- Cache results of expensive function calls.

Example:

```
from functools import lru_cache
@lru_cache(maxsize=128)
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
print(fib(10))    # 55
```

79. BINARY, OCTAL, HEXADECIMAL

Example:

```
a = 255
print(bin(a))    # 0b11111111
print(oct(a))    # 0o377
print(hex(a))    # 0xff
```

80. BITWISE OPERATORS

Example:

```
a, b = 5, 3     # 101, 011
print(a & b)    # 1   (AND)
print(a | b)    # 7   (OR)
print(a ^ b)    # 6   (XOR)
print(~a)       # -6  (NOT)
print(a << 1)   # 10  (Left shift)
print(a >> 1)   # 2   (Right shift)
```

81. PASS, CONTINUE, BREAK

Example:

```
# pass -> placeholder
for i in range(3):
    pass

# continue -> skip current iteration
for i in range(5):
    if i == 2: continue
    print(i)    # 0 1 3 4

# break -> exit loop
for i in range(5):
    if i == 3: break
    print(i)        # 0 1 2
```

82. PRACTICE ROADMAP

- [x] Learn Basics
- [x] Practice Daily
- [x] Build Mini Projects
- [x] Explore Libraries
- [x] Solve Problems
- [x] Read Documentation
- [x] Stay Consistent & Curious!

TIPS: Practice regularly, build small projects, and refer official Python documentation.

Monika Barde